

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PASHCHIMANCHAL CAMPUS

A Minor Project Report
On
EventX - One Place For All Events

Submitted By:

Aashish Adhikari (PAS079BCT003)
Ashim Shrestha (PAS079BCT007)
Sagun Paudel (PAS079BCT032)
Saugat Adhikari (PAS079BCT036)

Submitted To:

Department of Electronics and Computer Engineering
Pashchimanchal Campus
Pokhara, Nepal

May 13, 2026



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PASHCHIMANCHAL CAMPUS

A Minor Project Report
On
EventX - One Place For All Events

Submitted By:
Aashish Adhikari (PAS079BCT003)
Ashim Shrestha (PAS079BCT007)
Sagun Paudel (PAS079BCT032)
Saugat Adhikari (PAS079BCT036)

Submitted To:
Department of Electronics and Computer Engineering
Pashchimanchal Campus
Pokhara, Nepal

In partial fulfillment for the award of a Bachelor's degree in Computer Engineering

Under the Supervision of
Asst. Prof. Nabin Lamichhane

May 13, 2026

DECLARATION

We hereby declare that the report of the project entitled **"EventX"**, which is being submitted to the Department of Electronics and Computer Engineering, IOE, Pashchimanchal Campus, in partial fulfillment of the requirements for the award of the Degree of Bachelor in Computer Engineering, is a bona fide report of the work carried out by us.

The materials contained in this report have not been submitted to any university or institution for the award of any degree. We are the authors of this complete work, and no sources other than those listed in the references have been used in this work.

Aashish Adhikari (PAS079BCT003) _____

Ashim Shrestha (PAS079BCT007) _____

Sagun Paudel (PAS079BCT032) _____

Saugat Adhikari (PAS079BCT036) _____

Date: May, 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the Institute of Engineering for acceptance a project report entitled **"EventX"**, submitted by **Aashish Adhikari, Ashim Shrestha, Sagun Paudel, and Saugat Adhikari**, in partial fulfillment of the requirements for the Bachelor's Degree in Computer Engineering.

Supervisor:
Asst. Prof. Nabin Lamichhane
Department of Electronics and Computer Engineering
Institute of Engineering, Pashchimanchal Campus

External Examiner:

Head of Department:
Asst. Prof. Hom Nath Tiwari
Department of Electronics and Computer Engineering
Institute of Engineering, Pashchimanchal Campus

DATE OF APPROVAL: May, 2026

COPYRIGHT

The authors hereby grant permission to the Library and the Department of Electronics and Computer Engineering, Pashchimanchal Campus, Institute of Engineering, to make this project report publicly accessible for purposes of academic inspection and scholarly reference.

Furthermore, the authors consent that authorization for substantial reproduction of this report, strictly for academic and research-related use, may be granted by the supervising faculty members responsible for the project work herein documented, or, in their absence, by the Head of the Department under which the project was undertaken.

It is expressly understood that appropriate acknowledgment shall be accorded to the authors of this report and the Department of Electronics and Computer Engineering, Pashchimanchal Campus, Institute of Engineering, in any academic or scholarly use of the contents herein.

Any reproduction, publication, or utilization of this report or its substantial parts for commercial purposes or financial gain is strictly prohibited without the prior written consent of both the Department of Electronics and Computer Engineering, Pashchimanchal Campus, Institute of Engineering, and the authors.

All inquiries regarding permission for copying, reproduction, or other use of this material, in whole or in part, should be directed to:

Head of the Department

Department of Electronics and Computer Engineering

Institute of Engineering, Pashchimanchal Campus

ACKNOWLEDGEMENT

This project report entitled **"EventX"** has been prepared in partial fulfillment of the requirements for the Bachelor's Degree in Computer Engineering. We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Institute of Engineering, Pashchimanchal Campus, for providing us with the opportunity to work on this project as a part of our academic curriculum.

We are deeply thankful to our supervisor, **Asst. Prof. Nabin Lamichhane**, for his valuable guidance, continuous support, and constructive suggestions throughout the development and documentation of this project. His supervision helped us improve our understanding of system design, project planning, and proper documentation.

We would also like to extend our gratitude to the project coordinator, faculty members, and the Department of Electronics and Computer Engineering for their support and encouragement during the project period. Their feedback and guidance played an important role in completing this project successfully.

Finally, we would like to thank our friends, classmates, and everyone who directly or indirectly supported us during this project. The experience of working on EventX has enhanced our technical knowledge, teamwork, communication skills, and confidence in handling real-world software engineering projects.

Aashish Adhikari (PAS079BCT003)

Ashim Shrestha (PAS079BCT007)

Sagun Paudel (PAS079BCT032)

Saugat Adhikari (PAS079BCT036)

ABSTRACT

EventX is a web-based event management system designed to simplify and improve the process of organizing, managing, and viewing institutional events. The system provides a centralized platform where students, faculty admins, club admins, and super admins can interact with event-related information in an organized and efficient manner. It helps reduce manual effort, scheduling confusion, and communication gaps that commonly occur when events are managed through scattered or traditional methods.

The application includes key features such as user authentication, role-based access control, event creation, event approval, calendar-based event viewing, event search, timetable management, notification handling, and conflict checking. Faculty admins can create and manage class-related events, club admins can submit club events for approval, and super admins can approve or reject events before they are published. Students can view approved events, access timetables, search for events, and receive important notifications.

The system was analyzed and designed using Object-Oriented Analysis and Design (OOAD) principles. Various UML diagrams and an ER diagram were prepared to represent the structure, behavior, data flow, and interactions of the system. EventX aims to provide a reliable, user-friendly, and scalable solution for academic institutions by improving event coordination, reducing scheduling conflicts, and ensuring timely communication among users.

Keywords: Event Management System, Web Application, Event Calendar, Role-Based Access Control, Event Approval, Timetable Management, Notification System, Conflict Checking, OOAD, UML

TABLE OF CONTENTS

DECLARATION i

CERTIFICATE OF APPROVAL ii

COPYRIGHT iii

ACKNOWLEDGEMENT iv

ABSTRACT v

LIST OF TABLES ix

LIST OF FIGURES x

1 INTRODUCTION 1

1.1 Background . . . 1
1.2 Motivation . . . 1
1.3 Objective . . . 1
1.4 Problem statement . . . 1
1.5 Scope of Project . . . 2

2 LITERATURE REVIEW 3

3 REQUIREMENT ANALYSIS 5

3.1 Functional Requirements . . . 5
3.1.1 Authentication and Authorization . . . 5
3.1.2 Student Functionalities . . . 5
3.1.3 Faculty Admin Functionalities . . . 6
3.1.4 Club Admin Functionalities . . . 6
3.1.5 Super Admin Functionalities . . . 6
3.1.6 Event Management . . . 6
3.1.7 Event Viewing and Search . . . 7
3.1.8 Notifications and Alerts . . . 7
3.1.9 Reports and Analytics . . . 7
3.2 Non-Functional Requirements . . . 7

4 SYSTEM ARCHITECTURE 9

4.1 Theoretical Background . . . 9
4.2 Architecture/Algorithms Used for the Project . . . 9

4.3 Use Case Diagram 10
4.4 Use Case Specifications 11
4.5 Class Diagram for Data 16
4.6 Object Diagram 17
4.7 ER Diagram 18
4.8 Sequence Diagram 19
4.9 Communication Diagram 20
4.10 Activity Diagram 21
4.11 State Diagram 23
4.12 Component Diagram 24
4.13 Package Diagram 24
4.14 Data Flow Diagram 25
4.14.1 DFD Level 0 25
4.14.2 DFD Level 1 26
4.15 Deployment Diagram 27

5 METHODOLOGY **29**
5.1 Requirements Gathering 29
5.2 Design 29
5.2.1 UI/UX Design 30
5.2.2 System Design 30
5.3 Development 30
5.4 Testing 30
5.5 Deployment 30

6 IMPLEMENTATION DETAILS **31**
6.1 Frontend Implementation 31
6.2 Backend Implementation 32
6.3 Database Schema 32
6.4 API and External Services 33
6.5 Development and Code Quality Tools 34
6.6 Testing Tools 34
6.7 Hosting and Deployment 34
6.8 System Workflow 35
6.9 Gantt Chart 35
6.10 Screenshots 36

7 RESULTS AND DISCUSSIONS **39**

8 FUTURE SCOPE **40**

9 CONCLUSION **41**

10 EPILOGUE **42**

REFERENCES **43**

LIST OF TABLES

4.1 Use Case UC1: Login / Authentication 11
4.2 Use Case UC2: View Event Calendar 12
4.3 Use Case UC3: Manage Class Events 13
4.4 Use Case UC4: Manage Club Events 14
4.5 Use Case UC5: Manage Users & Permissions 15

LIST OF FIGURES

4.1 System Architecture 9
4.2 Use Case Diagram for EventX 11
4.3 Class Diagram for EventX 17
4.4 Object Diagram for EventX 18
4.5 ER Diagram for EventX 19
4.6 Sequence Diagram for EventX 20
4.7 Communication Diagram for EventX 21
4.8 Activity Diagram for EventX 22
4.9 State Diagram for EventX 23
4.10 Component Diagram for EventX 24
4.11 Package Diagram for EventX 25
4.12 Dataflow Diagram Level 0 for EventX 26
4.13 Dataflow Diagram Level 1 for EventX 27
4.14 Deployment Diagram for EventX 28
5.1 Agile Model 29
6.1 Database Schema of EventX 33
6.2 Project Timeline 35
6.3 Login Page 36
6.4 Home Page 36
6.5 Event Details Page 37
6.6 Club Admin Dashboard 37
6.7 Faculty Admin Dashboard 38
6.8 Super Admin Dashboard 38

1. INTRODUCTION

1.1. Background

EventX is an event management web application that provides a centralized, calendar-based platform for students to stay informed about all college club activities as well as class-specific events such as exams, assignments, deadlines, and timetables. The success of any event relies heavily on proper organization and timely communication between the organizers and students. However, many colleges lack a single platform where students can access both academic and extracurricular event information, leading to missed updates and confusion. This project intends to create a unified, web-based application to capture, track, and display all college events, including those from clubs and the institutional faculty.

1.2. Motivation

Students frequently miss important academic deadlines or club events due to scattered communication channels and lack of coordination. Similarly, event organizers face challenges in managing schedules and avoiding overlaps. The motivation behind EventX is to simplify event communication and management by offering a centralized, role-based system that ensures better organization and improved participation.

1.3. Objective

To develop a centralized web-based platform for managing curricular and extracurricular events at college.

1.4. Problem statement

In most of the colleges, event-related information, both from clubs and faculty, such as exams, assignments, deadlines, and timetables, is scattered across notice boards, social media, and messaging groups. This fragmented approach often leads to missed announcements, overlapping of the events, unmanaged time and schedule. Additionally, there is no single platform where all the clubs and faculties can publish events and students can get information about the events. This lack of a centralized

role-based event management system negatively impacts student participation and academic planning, highlighting the need for an integrated event management solution.

1.5. Scope of Project

The EventX system is designed for use within a college environment and supports multiple user roles including students, faculty admins, club admins, and super admins. It allows event creation, updating, deletion, and viewing through a calendar interface. The system includes features such as event filtering, notifications, and timetable management. However, the scope is limited to institutional use and does not cover large-scale public event integration and management systems.

2. LITERATURE REVIEW

The importance of managing activities within the university has grown significantly, whether it is curricular or extracurricular. Several traditional methods to convey event information like bulletin boards, physical classroom announcements, etc have resulted in fragmented communication and low participation. Our research in multiple studies suggested the implementation of event calendars and online management dashboards to enhance information accessibility, thus improving overall college event management. This digital event management platform will help organize event details and notifications in one centralized interface. Similar centralized event management approaches have been shown to improve the coordination and accessibility of institutional activities [1].

2.1. Existing Event Management Practices in Colleges

In most engineering colleges under Tribhuvan University, the information about the events are primarily disseminated through classroom announcements, notice boards and informal messaging platforms such as Messenger and WhatsApp. While these methods are simple, they include problems like failing to reach all intended participants on time whenever there are any changes to event details like date changes, venue updates, etc.[2] Due to lack of centralized information, students frequently miss events and teachers or clubs face difficulties to coordinate schedules and avoid event conflicts. From our observations, overlapping events are common in most of the colleges. For example, club activities and internal assessments are planned for the same day and time, which lowers participation in club activities.[3] These practical difficulties highlight the need of a centralized event management designed specially for educational institutions. Prior studies on academic institutions report that the absence of centralized event platforms often leads to scheduling conflicts and reduced student participation [4]. Furthermore, research on campus event management systems has demonstrated that web-based solutions significantly reduce scheduling conflicts and improve event visibility among students [5]. Real-time notification mechanisms have also been identified as a critical factor in improving student awareness and participation in institutional events [6].

2.2. Review of Existing Digital Event Management Systems

For scheduling and reminders, digital event management systems like Google Calendar, Eventbrite, and Microsoft Outlook are widely used worldwide. For example, Google Calendar allows users to create, update and manage all your tasks and also set reminders. However, despite offering extensibility through developer add-ons, such platforms are generic in nature and do not support university-specific

roles like students, teachers, clubs and administrators [7, 8]. Furthermore, they also lack approval systems, making them unsuitable for managing official college events. The same is true for Eventbrite which focuses on public and commercial events but not academic events. Similarly, Outlook Calendar is also like Google Calendar which is effective for personal scheduling but it does not provide university-events visibility in a structured manner.

2.3. Research Gap and Relevance of the Proposed System

The existing digital event management systems are not designed to address the organizational structure of the academic institutions. They don't distinguish different user roles such as students, teachers, clubs, and admins due to which important aspects such as event approval, authenticity, and accountability are often overlooked. Studies on role-based access control in academic information systems confirm that clearly defined user roles are essential for maintaining event authenticity and administrative accountability in institutional platforms [9]. Thus, our proposed Event Manager aims to address these gaps by incorporating a unified calendar that displays both general and class events, role-based event workflow and approval processes for club-organized events. So it is important to align digital platforms with institutional roles and approval structures. Research has further shown that unified digital platforms tailored to academic workflows lead to measurably higher student participation and smoother event coordination across departments [10]. This project seeks to improve the overall effectiveness of managing the event within the university setting by aligning system functionality with actual university practices.

3. REQUIREMENT ANALYSIS

Requirement analysis defines the functional and non-functional requirements of the EventX system. These requirements describe the expected behavior, features, constraints, and performance standards necessary for the successful operation of the platform.

3.1. Functional Requirements

Functional requirements describe the core functionalities and operations that the EventX system must perform.

3.1.1. Authentication and Authorization

The system provides secure user authentication and role-based authorization mechanisms. Users are required to sign in using valid credentials before accessing protected functionalities of the platform. The system verifies user credentials using the user database and maintains authenticated sessions securely.

The platform supports multiple user roles including Student, Faculty Admin, Club Admin, and Super Admin. Each role have different access privileges and permissions based on system requirements. Unauthorized users are not allowed to access restricted functionalities or modify protected data.

3.1.2. Student Functionalities

Students are able to access and view all approved academic and extracurricular events through a centralized calendar interface. The system allows students to view event details such as title, description, date, time, venue, and organizer information.

Students are also able to search and filter events based on event category, organizer, and date range. In addition, the system provides notifications and reminders for upcoming events and important announcements.

3.1.3. Faculty Admin Functionalities

Faculty Admins are responsible for managing academic and class-related events. The system allows Faculty Admins to create, edit, update, and delete class events created by themselves.

Faculty Admins are also able to view academic events using the calendar-based interface and manage timetable-related information within the system.

3.1.4. Club Admin Functionalities

Club Admins are responsible for managing club-related activities and events. The system allows Club Admins to create, update, and delete club events associated with their respective clubs.

The platform restricts Club Admins from modifying events created by Faculty Admins or other clubs to maintain proper access control and data integrity.

3.1.5. Super Admin Functionalities

The Super Admin has complete administrative control over the system. The system allows the Super Admin to view, modify, approve, or delete any event within the platform.

The Super Admin also manages users, assign roles, define permissions, and monitor overall system activities. The system additionally supports conflict detection for events based on overlapping date, time, and venue information.

3.1.6. Event Management

The system supports Create, Read, Update, and Delete (CRUD) operations for event management. Faculty Admins and Club Admins are able to manage events according to their assigned roles and permissions.

All event-related information are securely stored within the database and displayed through a calendar-based user interface.

3.1.7. Event Viewing and Search

The system displays approved events in daily, weekly, and monthly calendar views. Users are able to select specific dates to view scheduled events for that particular day.

The platform provides filtering and search functionalities based on event category, organizer, and date range to improve accessibility and usability.

3.1.8. Notifications and Alerts

The system provides notifications whenever new events are added or updated. Users receives reminders and alerts for upcoming events based on their preferences and schedules.

3.1.9. Reports and Analytics

The system generates basic reports and analytics related to event management activities. These reports include information such as the total number of events created, user activity records, and event participation trends.

3.2. Non-Functional Requirements

Non-functional requirements define the quality attributes and operational standards of the EventX system.

Usability: The system provides a simple, intuitive, and user-friendly interface to ensure easy navigation and accessibility for all users.

Performance: The application provides fast response times and efficient loading under normal operating conditions.

Scalability: The system supports a growing number of users, events, and database records without significant performance degradation.

Security: The platform implements secure authentication, encrypted password storage, and HTTPS-based communication to protect user data and system resources.

Maintainability: The system architecture and codebase follow modular development practices to

simplify maintenance, updates, and future enhancements.

Reliability: The system ensures consistent availability and accurate handling of event-related information.

Compatibility: The web application supports modern web browsers and responsive access across multiple device sizes.

Code Quality: The development process includes linting, formatting, type checking, and automated validation mechanisms to maintain software quality and reliability.

4. SYSTEM ARCHITECTURE

This section presents the overall design of the EventX system through UML and data flow diagrams. The diagrams explain the system structure, user interactions, data movement, and deployment setup for the college event management platform. The system supports four main roles: Student, Faculty Admin, Club Admin, and Super Admin.

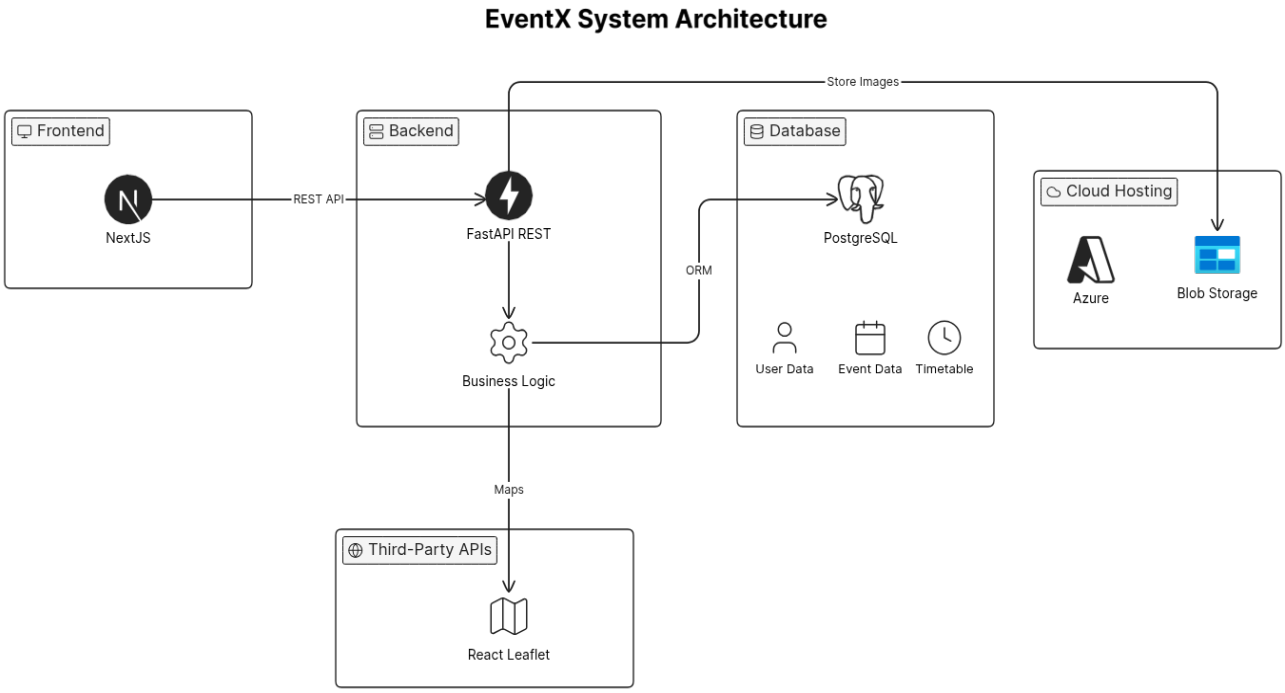


Figure 4.1: System Architecture

4.1. Theoretical Background

EventX is designed as a web-based, role-driven event management system. It follows object-oriented design and separates the responsibilities of users, events, notifications, and timetables so that academic and extracurricular activities can be managed in an organized way. The use case model shows the major functions such as login, calendar viewing, event management, and user administration.

4.2. Architecture/Algorithms Used for the Project

The system follows a client-server architecture. The frontend is used for user interaction, while the backend processes requests, applies role-based access control, and stores or retrieves data from the

database. The main workflow includes authentication, event creation or viewing, notification delivery, and user management.

4.3. Use Case Diagram

The Use Case Diagram of EventX shows how four actors i.e. Student, Faculty Admin, Club Admin, and Super Admin interact with the system. Students first login to the system, then view the event calendar. They can also search and filter class events or club events, view event details, receive notifications about upcoming events and view class timetables. Faculty admins authenticate themselves and create, update and delete class events, upload or update timetables. Club Admins use the system to create, manage, and update club events, and publish event information to the calendar. Admins oversee the overall operation of the system by managing users and permissions, controlling access. Super Admin oversees the overall system, manages user and permissions and controls all the club and class events. The system allows to filter and search events, timetable management, and notification delivery, enabling smooth coordination of the academic and extracurricular activities the user interacts with.

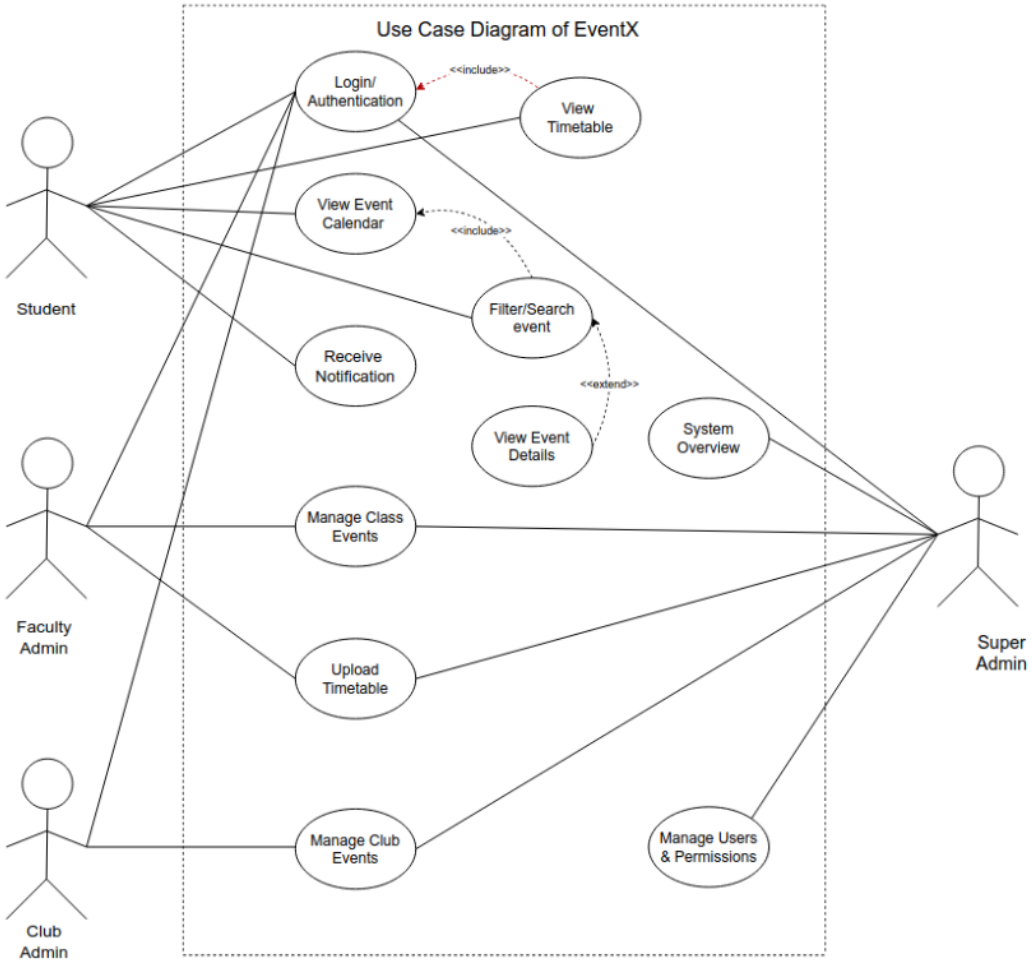


Figure 4.2: Use Case Diagram for EventX

4.4. Use Case Specifications

Table 4.1: Use Case UC1: Login / Authentication

Use Case ID	UC1
Use Case Name	Login / Authentication
Primary Actor	Student, Faculty Admin, Club Admin, Super Admin
Stakeholders & Interests	Student: Wants secure, quick access to events and timetables. Faculty Admin: Wants authenticated access to manage class events. Club Admin: Wants authenticated access to manage club events. Super Admin: Wants only authorised users to access the system.
Preconditions	User has a registered account in EventX.

Success Guarantee (Post-conditions)	User is authenticated, a session is created, and the user is directed to their role-based dashboard.
Main Success Scenario	1. User opens the EventX login page. 2. User enters username/email and password. 3. System validates the credentials. 4. System identifies the user's role. 5. System creates a session and redirects the user to their dashboard.
Extensions (Alternative Flows)	3a. Invalid credentials: System shows an error; user may retry. 3b. Account locked: Locked after 5 failed attempts; user must reset password. 2a. Forgot password: System emails a reset link; user resets and logs in.
Special Requirements	- Passwords stored as salted hashes. - Session expires after 30 minutes of inactivity. - Login page must work on mobile devices.
Technology & Data Variations	- Credentials entered via web form or mobile app. - Future: institutional SSO/OAuth support.
Frequency of Occurrence	Multiple times daily per user.

Table 4.2: Use Case UC2: View Event Calendar

Use Case ID	UC2
Use Case Name	View Event Calendar
Primary Actor	Student
Stakeholders & Interests	Student: Wants all upcoming class and club events in one view. Faculty Admin: Wants class events visible to students after publishing. Club Admin: Wants club events to appear on the calendar promptly. Super Admin: Wants the calendar to always show accurate information.
Preconditions	User is logged in (UC1). At least one published event exists.
Success Guarantee (Post-conditions)	Event calendar is displayed with all published events for the selected period.
Main Success Scenario	1. Student navigates to the Event Calendar. 2. System retrieves all published events for the current month. 3. System displays events in a monthly calendar view. 4. Student can navigate between months. 5. Student may search or filter events. 6. Student clicks an event to view its details.

Extensions (Alternative Flows)	2a. No events for the period: System displays 'No events found.' 4a. Student switches view: System re-renders in weekly or daily mode. *a. Data fails to load: System shows an error with a retry option.
Special Requirements	- Calendar must be responsive on mobile screens. - Class and club events must be visually distinguished.
Technology & Data Variations	- Calendar rendered with a frontend library (e.g., FullCalendar.js). - Event data fetched via REST API.
Frequency of Occurrence	Multiple times daily by students.

Table 4.3: Use Case UC3: Manage Class Events

Use Case ID	UC3
Use Case Name	Manage Class Events
Primary Actor	Faculty Admin
Stakeholders & Interests	Faculty Admin: Wants to create, edit, and delete class events easily. Student: Wants accurate class events on the calendar in real time. Super Admin: Wants class events properly categorised and auditable. Institution: Wants academic events coordinated to avoid conflicts.
Preconditions	Faculty Admin is logged in (UC1) and has class event management permissions.
Success Guarantee (Postconditions)	Class event is saved. Calendar is updated immediately. Students are notified.
Main Success Scenario	1. Faculty Admin opens 'Manage Class Events'. 2. System shows the list of existing class events. 3. Faculty Admin selects 'Create New Event'. 4. Faculty Admin fills in: title, date, time, venue, description, target audience. 5. System validates the input. 6. System publishes the event to the calendar. 7. System notifies relevant students.
Extensions (Alternative Flows)	3a. Edit event: Faculty Admin selects an event, edits fields, saves; students re-notified. 3b. Delete event: Faculty Admin confirms deletion; event removed and students notified. 5a. Validation fails: System highlights errors; Faculty Admin corrects and resubmits.

Special Requirements	- Description field must support rich text. - System must warn of venue/time conflicts with existing events.
Technology & Data Variations	- Form submission with client-side and server-side validation. - File attachments (slides, agendas) can be added to an event.
Frequency of Occurrence	Several times per week during the academic term.

Table 4.4: Use Case UC4: Manage Club Events

Use Case ID	UC4
Use Case Name	Manage Club Events
Primary Actor	Club Admin
Stakeholders & Interests	Club Admin: Wants to create and promote club events easily. Student: Wants to discover and attend relevant club events. Super Admin: Wants club events to comply with institutional policy before going live. Institution: Wants extracurricular events coordinated with academic ones.
Preconditions	Club Admin is logged in (UC1) and has permissions to manage their club's events.
Success Guarantee (Postconditions)	Club event is saved and published to the calendar. Students are notified.
Main Success Scenario	1. Club Admin opens 'Manage Club Events'. 2. System shows existing events for the admin's club. 3. Club Admin selects 'Create New Club Event'. 4. Club Admin fills in: title, date, time, venue, description, registration info. 5. System validates the input. 6. System publishes the event to the calendar. 7. System notifies students.
Extensions (Alternative Flows)	3a. Edit event: Club Admin modifies fields and saves; students re-notified. 3b. Delete event: Club Admin confirms; event removed and participants notified. 6a. Approval required: Event set to 'Pending'; Super Admin approves/rejects; Club Admin notified. 5a. Validation fails: System highlights errors; Club Admin corrects and resubmits.

Special Requirements	- Club Admin can only manage events for their own club. - A venue is required before an event can be published.
Technology & Data Variations	- Rich text editor for descriptions. - Promotional images or documents can be attached.
Frequency of Occurrence	Several times per month during the academic year.

Table 4.5: Use Case UC5: Manage Users & Permissions

Use Case ID	UC5
Use Case Name	Manage Users & Permissions
Primary Actor	Super Admin
Stakeholders & Interests	Super Admin: Wants full control over user accounts and access rights. Faculty Admin / Club Admin: Want roles configured correctly to perform their tasks. Student: Wants their account active and correctly set up. Institution: Wants only authorised personnel in administrative roles.
Preconditions	Super Admin is logged in (UC1).
Success Guarantee (Post-conditions)	User accounts are created, updated, or deactivated. Role changes take effect immediately.
Main Success Scenario	1. Super Admin opens ‘User Management’. 2. System displays all users with their roles and status. 3. Super Admin selects a user or creates a new account. 4. Super Admin assigns a role (Student, Faculty Admin, Club Admin, Super Admin). 5. Super Admin sets or revokes specific permissions. 6. System saves changes; access rights update immediately. 7. System emails a welcome message if a new account was created.
Extensions (Alternative Flows)	3a. Deactivate account: System disables login and logs the action with a timestamp. 3b. Reactivate account: System restores access for the selected user. 5a. Permission conflict: System warns Super Admin; conflict must be resolved before saving. 3c. Bulk import: Super Admin uploads a CSV; system creates accounts in batch and reports errors.

Special Requirements	- All permission changes must be logged for audit purposes. - Bulk import of student accounts via CSV must be supported. - Role changes must take effect without the user needing to re-login.
Technology & Data Variations	- Role-based access control (RBAC) enforced at the application layer. - Bulk import via CSV with server-side row validation.
Frequency of Occurrence	At the start of each semester and on-demand.

4.5. Class Diagram for Data

The class diagram represents the static structure of the system. It shows the main classes in EventX, such as Calendar, Event, Notification, Timetable, Student, Faculty Admin, Club Admin, and Super Admin, along with the relationships between them. This diagram is useful for understanding how classes are organized and related in the system.

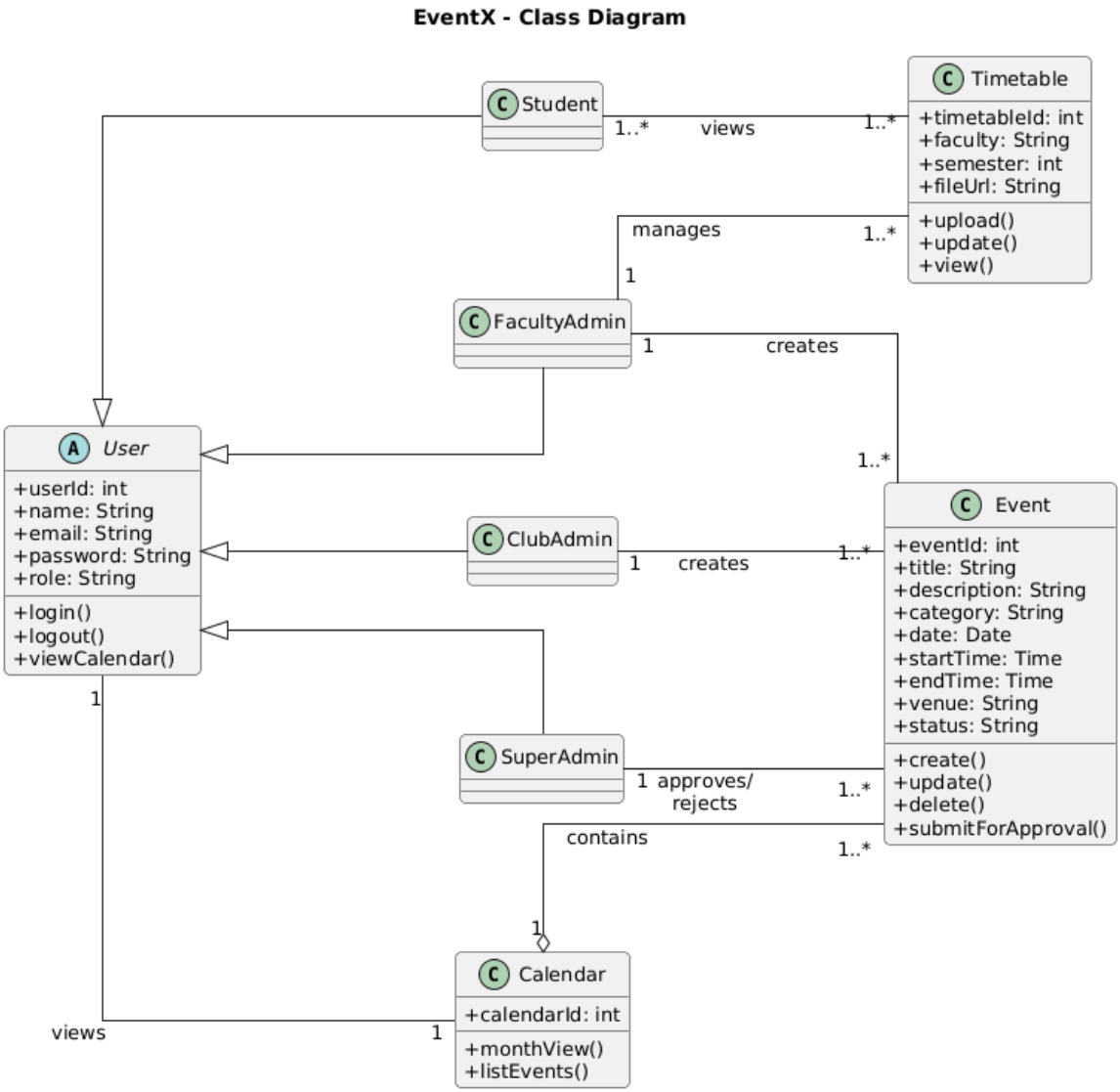


Figure 4.3: Class Diagram for EventX

4.6. Object Diagram

The object diagram shows the relationship between object instances at runtime. It provides a practical representation of how users, events, calendars, and timetables interact within the EventX system during execution.

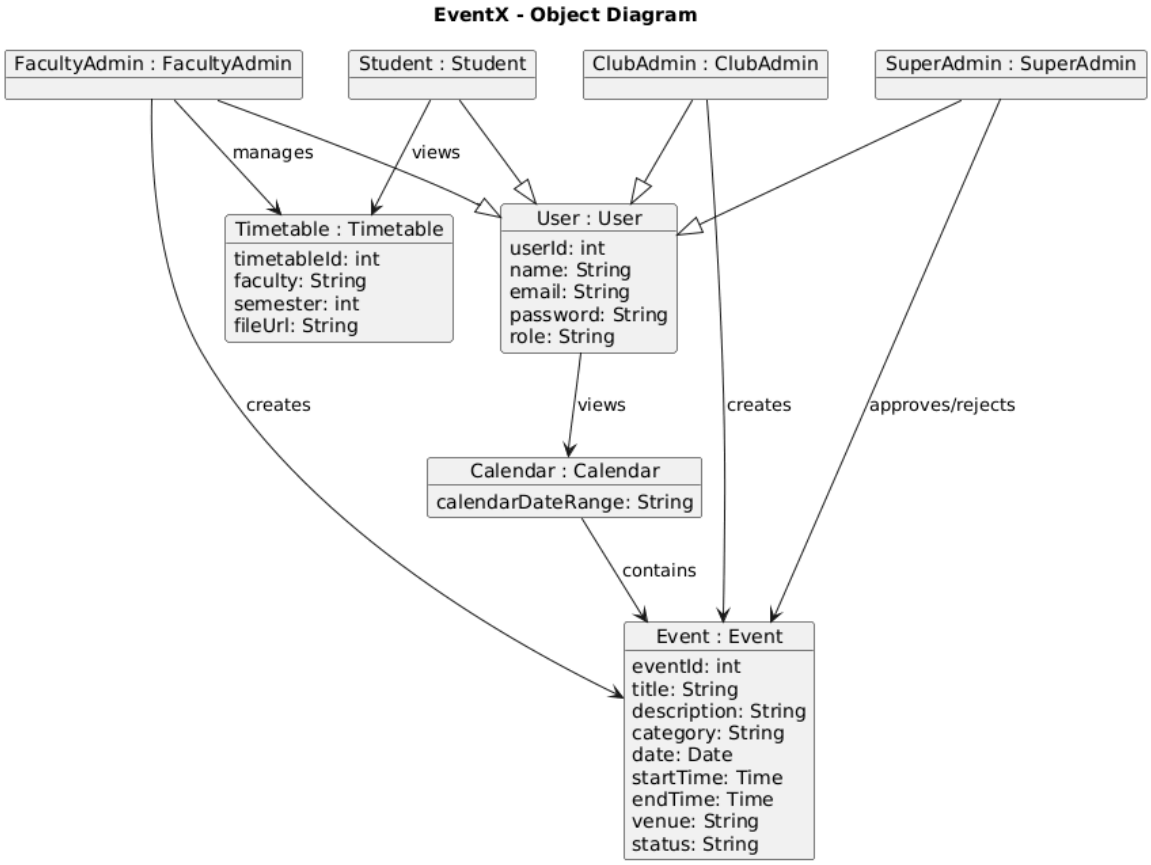


Figure 4.4: Object Diagram for EventX

4.7. ER Diagram

The ER diagram shows the database design of EventX. It defines the entities used in the system and the relationships among them, such as users, events, notifications, and timetables. This diagram helps explain how the system stores and connects data in the database.

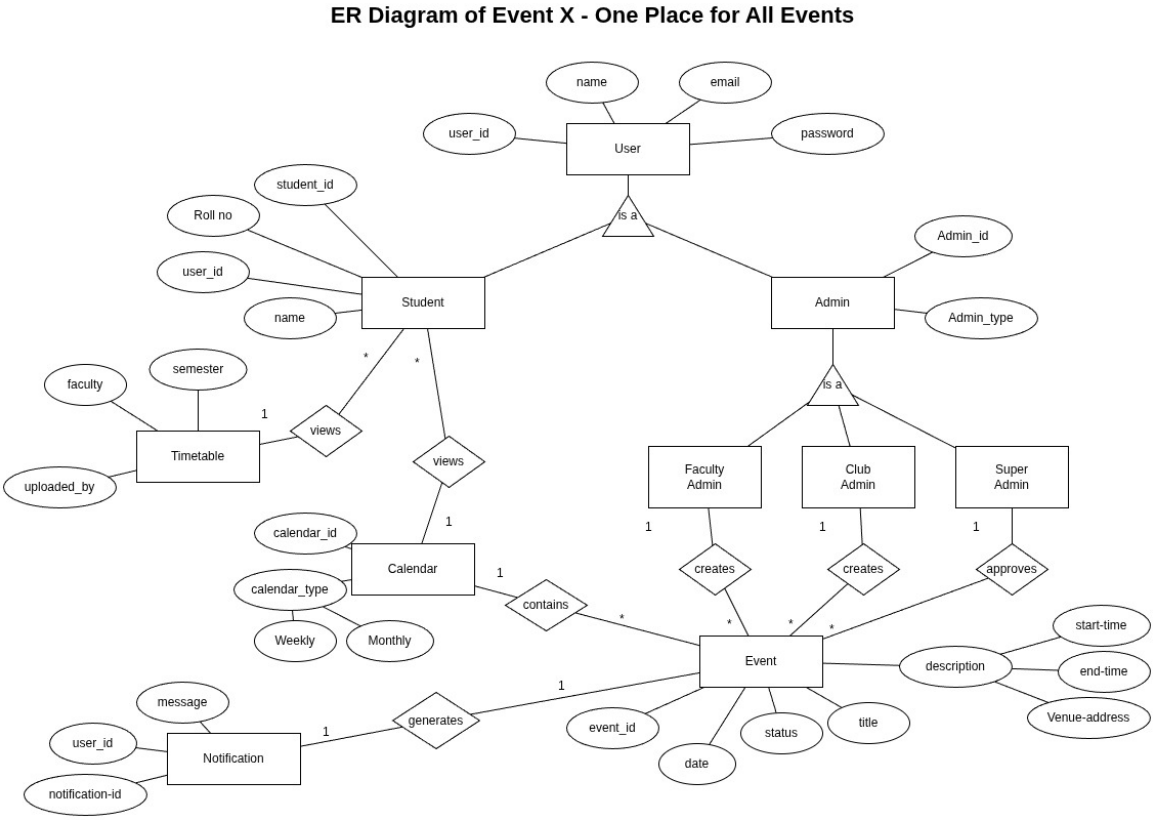


Figure 4.5: ER Diagram for EventX

4.8. Sequence Diagram

The sequence diagram shows the order of messages exchanged between the user, the frontend, the backend, and the database for a specific operation. It is useful to understand how the system responds step by step during actions such as login, event creation, and event viewing.

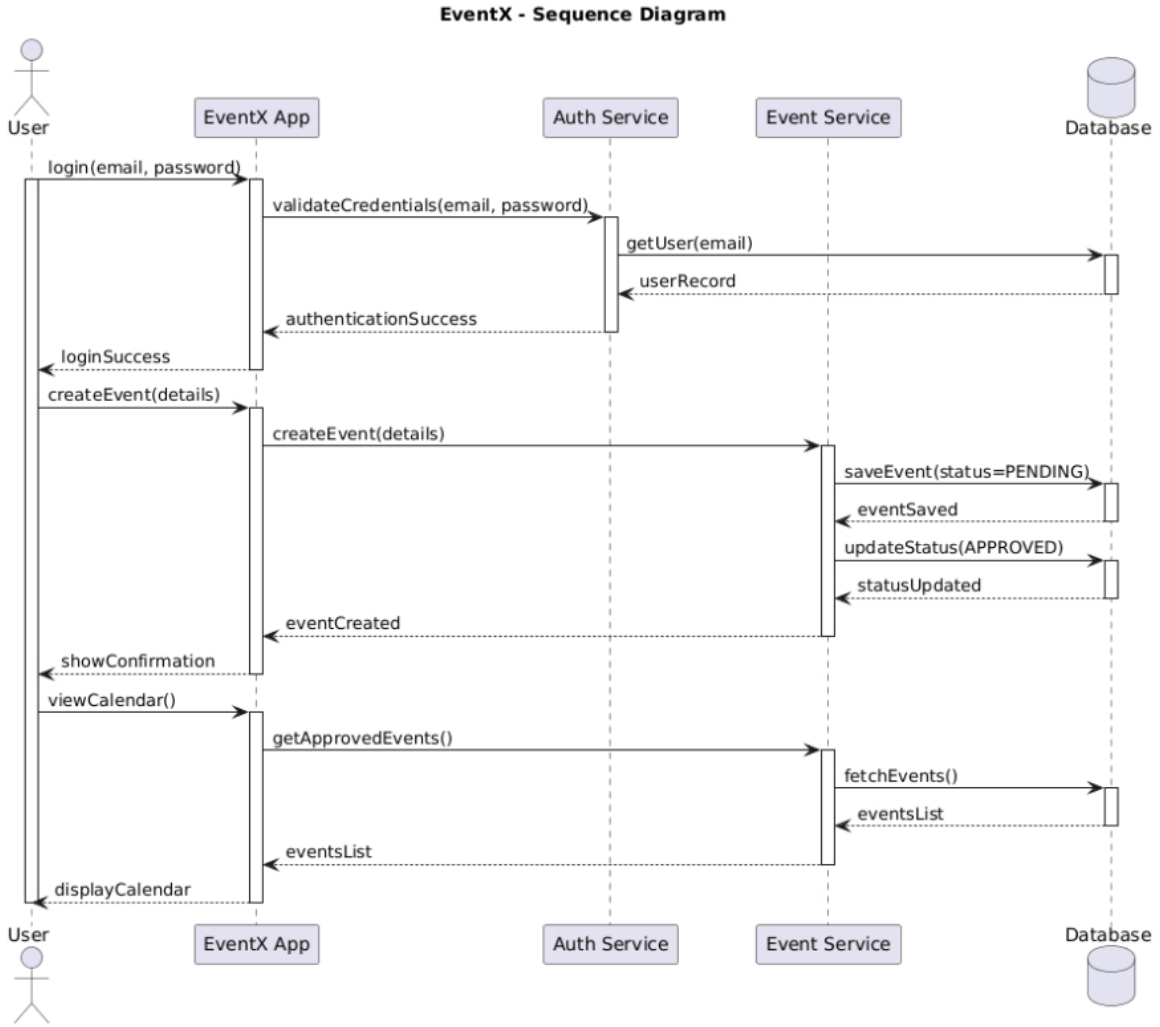


Figure 4.6: Sequence Diagram for EventX

4.9. Communication Diagram

The communication diagram shows how the different objects or components in EventX interact with each other and exchange messages. It focuses on the relationships and communication links between components rather than the time sequence of messages.

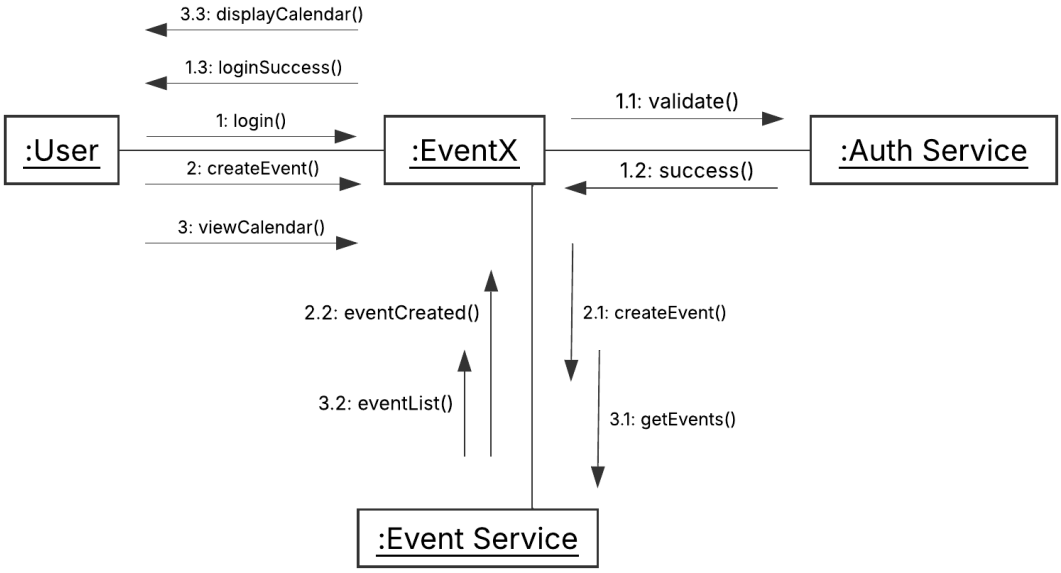


Figure 4.7: Communication Diagram for EventX

4.10. Activity Diagram

The activity diagram represents the flow of actions in the system. It shows the sequence of steps involved in processes such as login, event management, and calendar viewing, including decisions and transitions between activities.

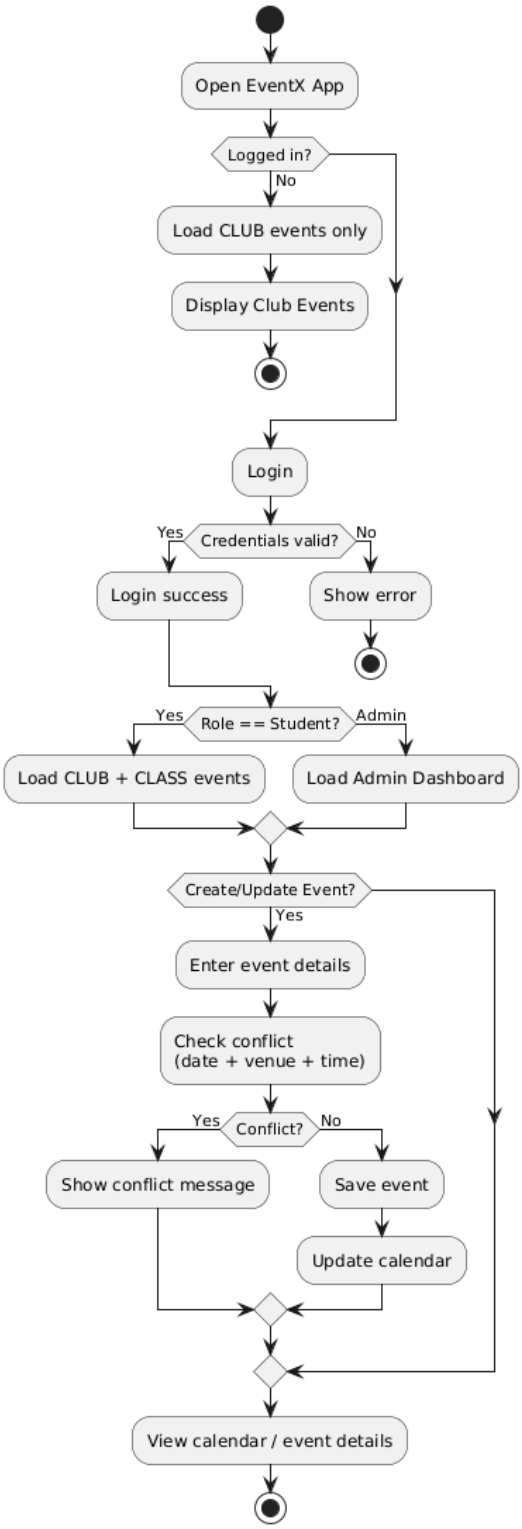


Figure 4.8: Activity Diagram for EventX

4.11. State Diagram

The state diagram describes the lifecycle of an event within the EventX system. It shows the different states of an event such as draft, validation, published, conflict, updated, and cancelled.

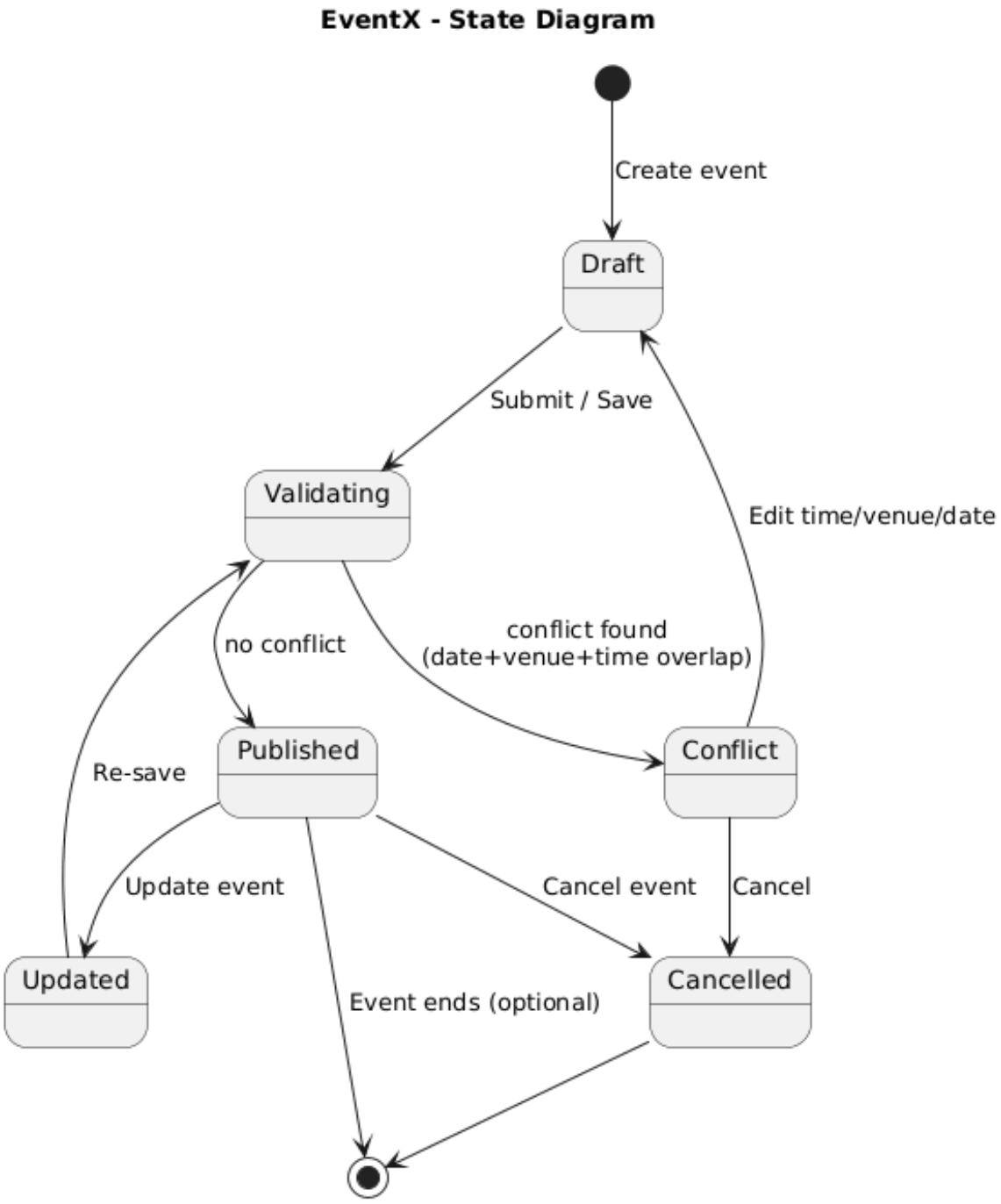


Figure 4.9: State Diagram for EventX

4.12. Component Diagram

The component diagram illustrates the major components of the EventX system and their interaction. It includes the frontend application, backend services, authentication module, event management module, timetable module, and database layer.

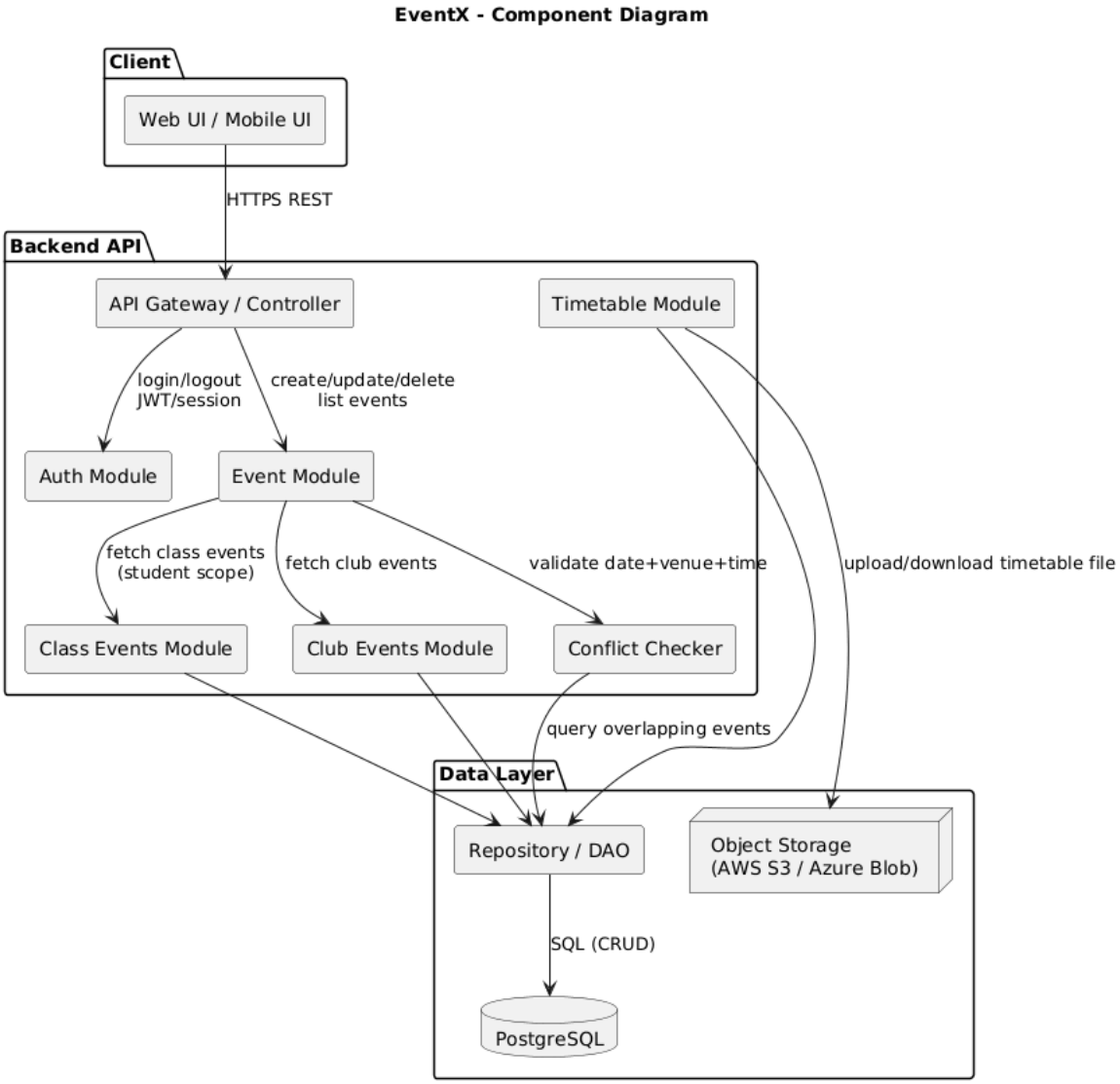


Figure 4.10: Component Diagram for EventX

4.13. Package Diagram

The package diagram represents the modular structure of the EventX system. It organizes the application into different layers such as presentation layer, API layer, business logic layer, and data access layer for better maintainability and scalability.

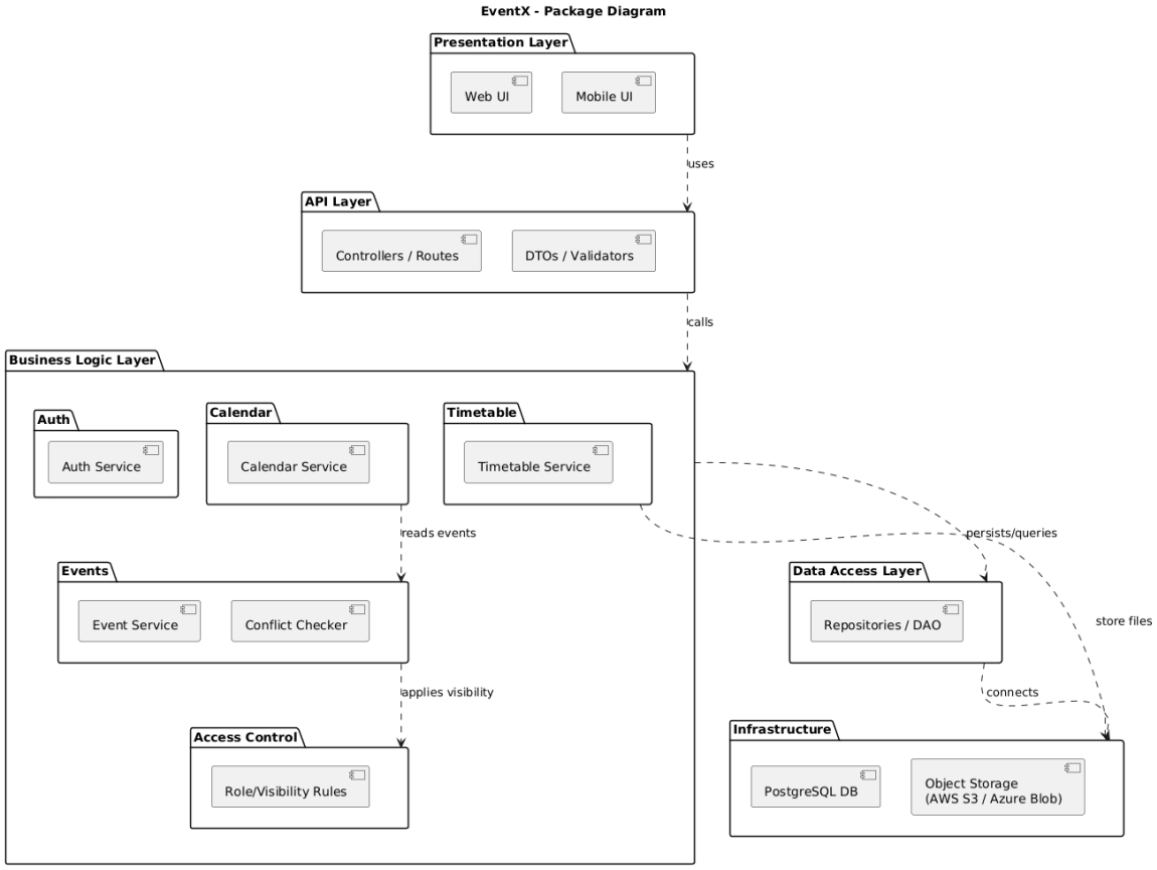


Figure 4.11: Package Diagram for EventX

4.14. Data Flow Diagram

The data flow diagram explains how information moves through the EventX system. It shows the flow of data between external users, processes, and data stores.

4.14.1. DFD Level 0

The Level 0 DFD provides the highest-level view of the system. It shows EventX as a single process and the interaction between the system and external entities such as students, faculty admins, club admins, and super admins.

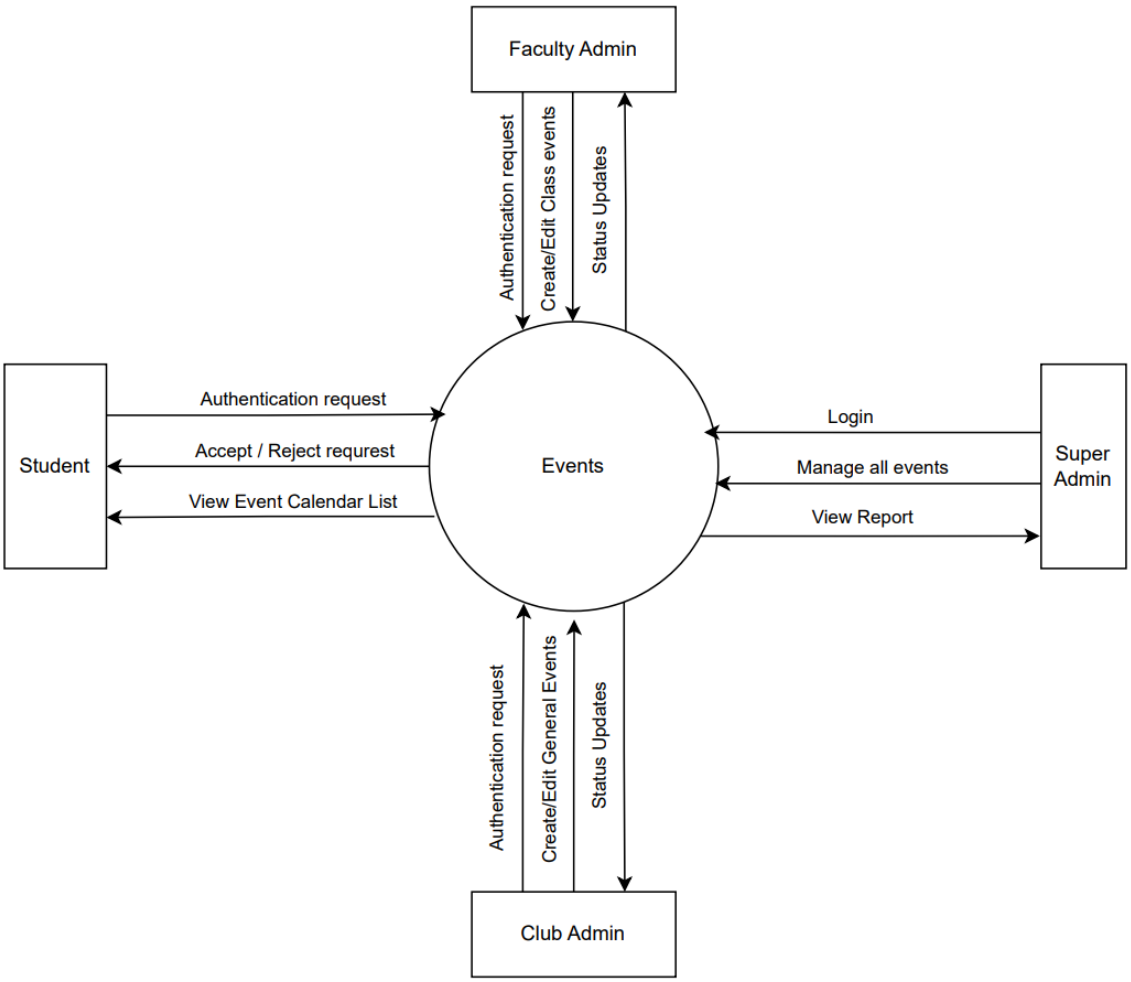


Figure 4.12: Dataflow Diagram Level 0 for EventX

4.14.2. DFD Level 1

The Level 1 DFD breaks the system into smaller processes such as authentication, event management, notification handling, and database interaction. It gives a more detailed view of how data is processed inside EventX.

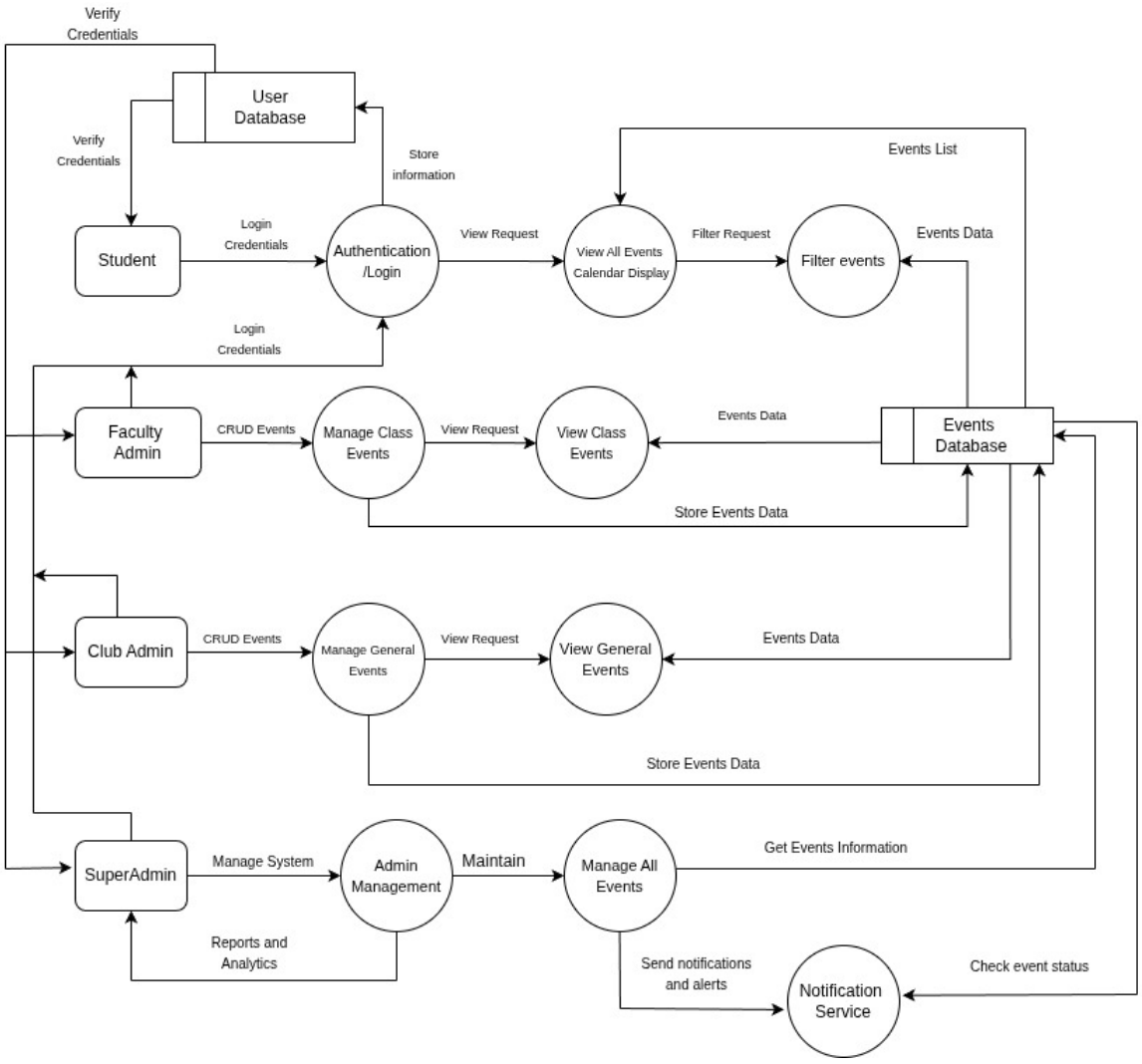


Figure 4.13: Dataflow Diagram Level 1 for EventX

4.15. Deployment Diagram

The deployment diagram shows how the EventX application is deployed across different environments. It represents the physical arrangement of the frontend, backend, and database components, along with their communication paths.

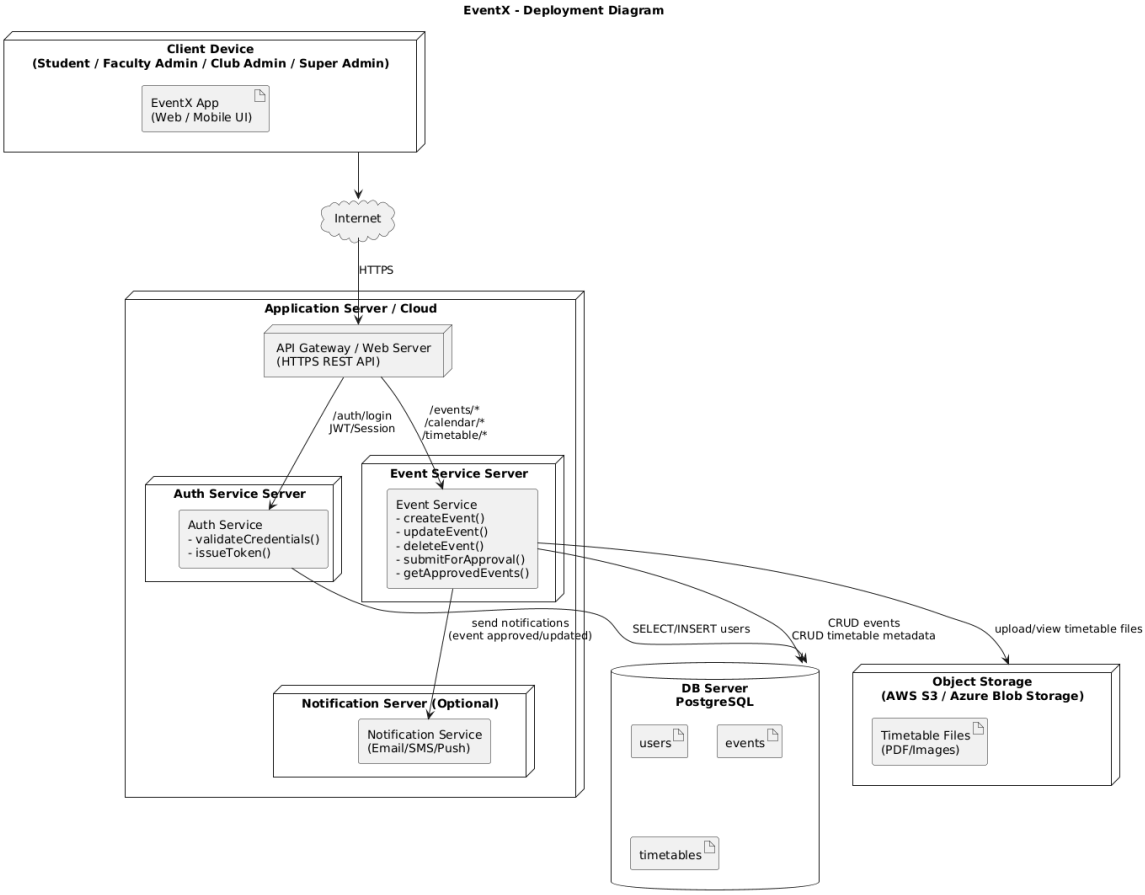


Figure 4.14: Deployment Diagram for EventX

5. METHODOLOGY

The development of the EventX app followed the Agile method, which emphasized iterative development, flexibility, and continuous feedback. The project was divided into multiple phases, each phase focusing on specific functionalities of the system. The use of an agile model allowed continuous improvement and feedback.

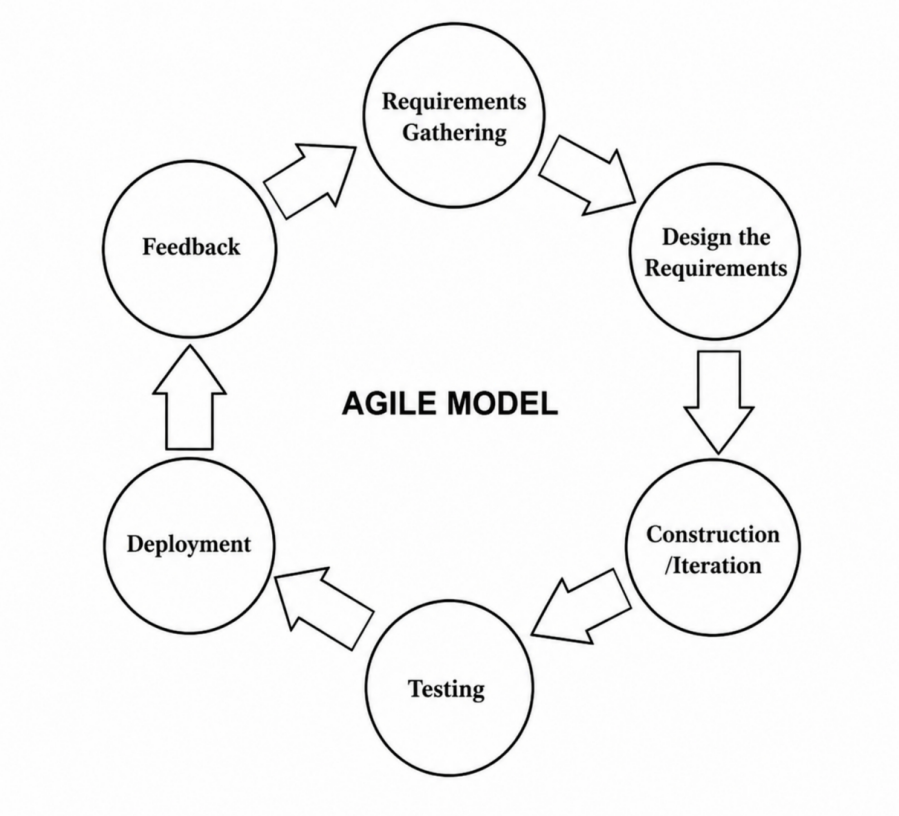


Figure 5.1: Agile Model

5.1. Requirements Gathering

The first phase of the project involved gathering requirements from the students and clubs who organize the events. This included conducting surveys, interviews, and focus groups to identify user needs and preferences. The requirements were gathered from students who regularly participated in college events and defined a problem statement. The requirements were then analyzed and prioritized.

5.2. Design

The design phase consisted of two main parts:

5.2.1. UI/UX Design

UI/UX design included designing the app interface such that the users will find the design pleasing and easy to navigate.

5.2.2. System Design

System design included specifying the database, user roles (admin, club, and user), event flow, and overall system architecture.

5.3. Development

The development phase included building the app in small sprints where each sprint focused on building and implementing specific features according to the priority defined in the backlog. The development was done using the JavaScript library Next.js, and the backend was developed using FastAPI in Python.

5.4. Testing

Testing of the product was held using tools like Selenium and Faker for frontend, whereas Postman and FastAPI Swagger were used for backend. The testing was done in two phases: unit testing and integration testing. The testing was done by the developers, which will be followed by the review of a supervisor.

5.5. Deployment

The final phase of the project involved deploying the app to a cloud server. The app was hosted on Vercel for frontend and Azure for backend, and the deployment process involved setting up the server, configuring the database, and ensuring that the app is accessible to users.

6. IMPLEMENTATION DETAILS

The EventX system is implemented as a modern web-based application designed to provide a centralized platform for managing academic and extracurricular events within a college. The implementation combines frontend technologies, backend services, database management systems, and deployment tools to ensure scalability, maintainability, and efficient system performance.

The overall system follows a client-server architecture where the frontend communicates with backend APIs to retrieve and manage event-related data. The backend processes requests, handles authentication and business logic, and interacts with the database for data storage and retrieval.

6.1. Frontend Implementation

The frontend of EventX is developed using **Next.js 16** with **TypeScript** to build a fast, responsive, and scalable user interface. Next.js provides server-side rendering and optimized routing, improving performance and user experience.

HTML, CSS, and JavaScript: These technologies are used as the foundational technologies for structuring, styling, and adding interactivity to the web application.

TypeScript: TypeScript is used to provide static typing and improve code maintainability, readability, and reliability during development.

Tailwind CSS v4: Tailwind CSS is used to create responsive and consistent user interfaces using utility-first styling techniques.

Zustand: Zustand is integrated for lightweight and efficient client-side state management throughout the application.

Radix UI: Radix UI components are used to develop accessible, reusable, and maintainable user interface components.

The frontend application is organized using the Next.js App Router structure, improving modularity and maintainability. Static resources are stored inside the `public/` directory, while application pages and components are maintained within the `src/app/` directory.

6.2. Backend Implementation

The backend services are developed using **FastAPI**, a modern Python framework designed for building secure and high-performance APIs. The backend handles user authentication, event management, timetable management, notification handling, and communication with the database.

FastAPI: FastAPI is used to create RESTful APIs with high performance and automatic API documentation support.

PostgreSQL: PostgreSQL is used as the primary relational database management system for securely storing user data, event information, timetables, and notifications.

Azure Blob Storage: Azure Blob Storage is used for secure and scalable cloud-based storage of event-related images and media files.

The backend APIs communicate with the frontend through HTTP requests and JSON-based data exchange.

6.3. Database Schema

The database schema for **EventX** is designed to manage users, faculties, clubs, events, and timetables in a structured way. All types of users, including students, faculty admins, club admins, and super admins, are stored in the users table and distinguished using the role attribute. The faculties table stores faculty or department information, while the clubs table stores details of different student clubs. Events are stored in the events table and are classified as either club_event or class_event using the event_type field. Club events are linked to clubs, while class events can be linked to faculties. The timetables table stores timetable images for specific faculties, batches, and sections. The relationships between tables ensure that users, events, and timetables are properly connected to their respective faculties or clubs.



Figure 6.1: Database Schema of EventX

6.4. API and External Services

The system integrates external services to provide additional functionality.

OpenStreetMap (OSM): OpenStreetMap is used to display event locations and provide map-based information within the platform.

6.5. Development and Code Quality Tools

To maintain code quality and support collaborative development, multiple development tools and automation practices are implemented.

Git: Git is used as the version control system for tracking source code changes and managing development history.

GitHub: GitHub is used for collaborative development, repository hosting, and project management.

ESLint: ESLint is used to identify and fix code quality and syntax issues during development.

Prettier: Prettier is used to maintain consistent code formatting throughout the project.

Husky: Husky is used to automate pre-commit hooks for linting, formatting, type checking, and build validation.

GitHub Actions: GitHub Actions is used to implement Continuous Integration (CI) workflows for automated testing and build verification.

Before each commit, automated checks such as formatting validation, linting, type checking, and production build verification are executed to ensure code reliability and maintain development standards.

6.6. Testing Tools

Different testing and validation tools are used to verify the functionality and reliability of the system.

Postman: Postman is used to test and validate backend API endpoints.

FastAPI Swagger UI: Swagger UI is used for interactive API testing and backend API documentation.

Selenium: Selenium is used for automated browser testing of application workflows and functionalities.

Faker: Faker is used to generate dummy data for testing and development purposes.

6.7. Hosting and Deployment

The EventX system is deployed using modern cloud and containerization technologies to ensure scalability and portability.

Vercel: Vercel is used for deploying and hosting the Next.js frontend application.

Microsoft Azure: Microsoft Azure is used for deploying and managing FastAPI backend services.

6.8. System Workflow

The system operates through the interaction of multiple interconnected modules. Users access the frontend application through a web browser, where they can view academic and extracurricular events through a centralized calendar interface.

The frontend communicates with backend APIs to perform operations such as authentication, event creation, timetable management, and event retrieval. The backend processes these requests and interacts with the PostgreSQL database for persistent data storage.

Faculty admins and club admins are responsible for creating and managing events, while students can access event schedules, timetables, and notifications. The synchronization between frontend services, backend APIs, and the database enables efficient communication and smooth operation of the entire system.

6.9. Gantt Chart

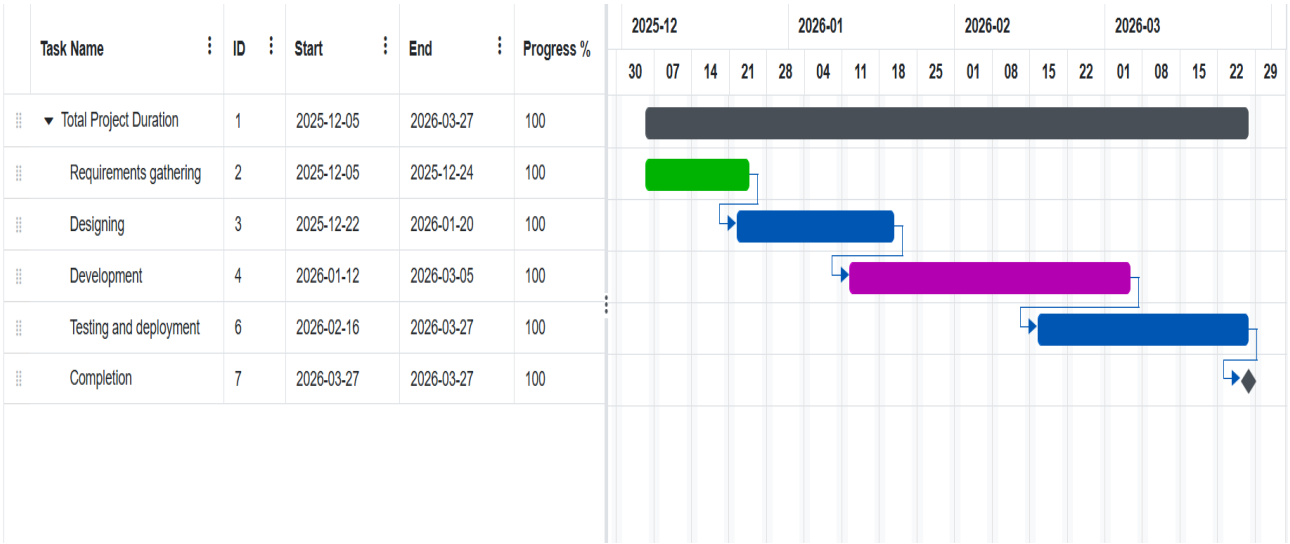


Figure 6.2: Project Timeline

6.10. Screenshots

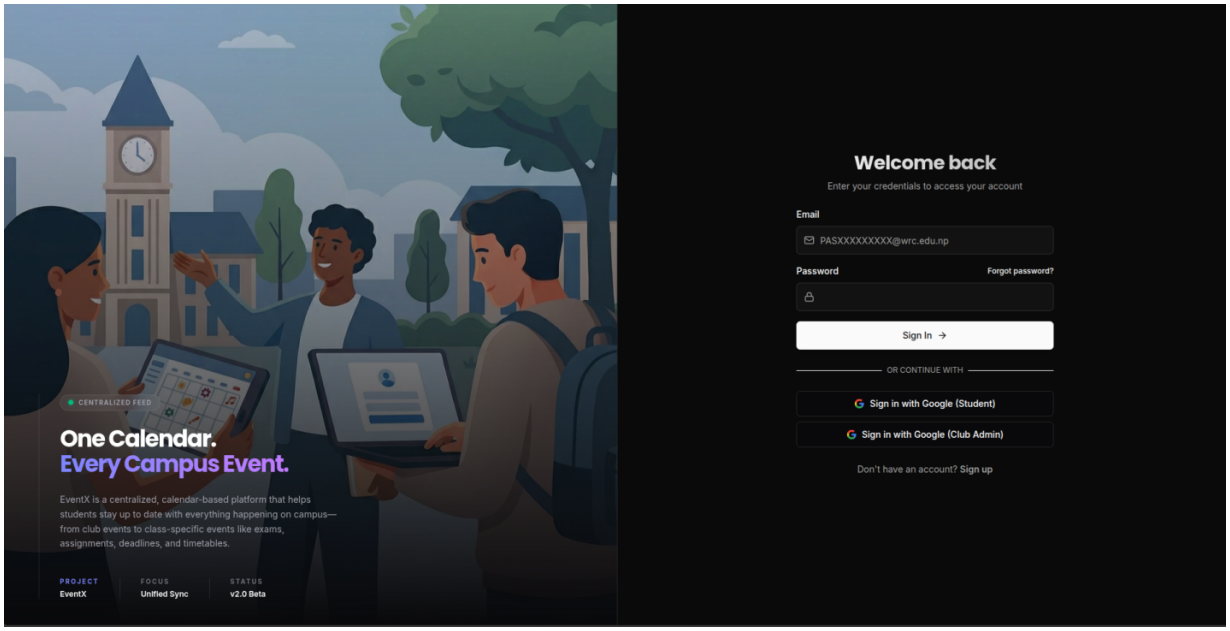


Figure 6.3: Login Page

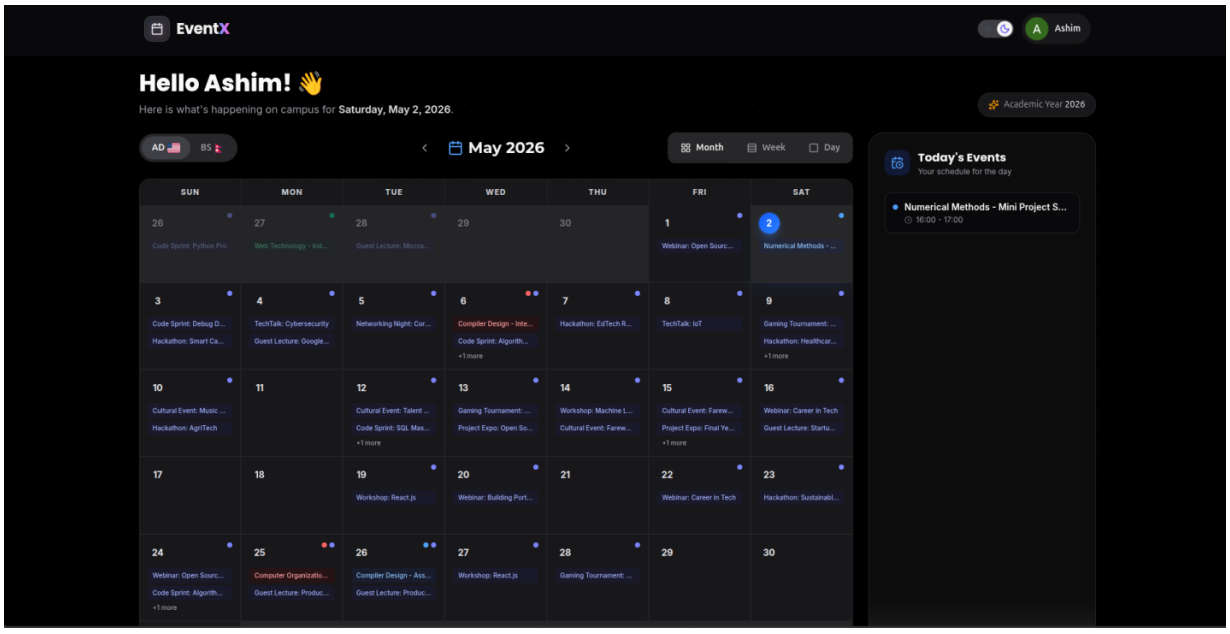


Figure 6.4: Home Page

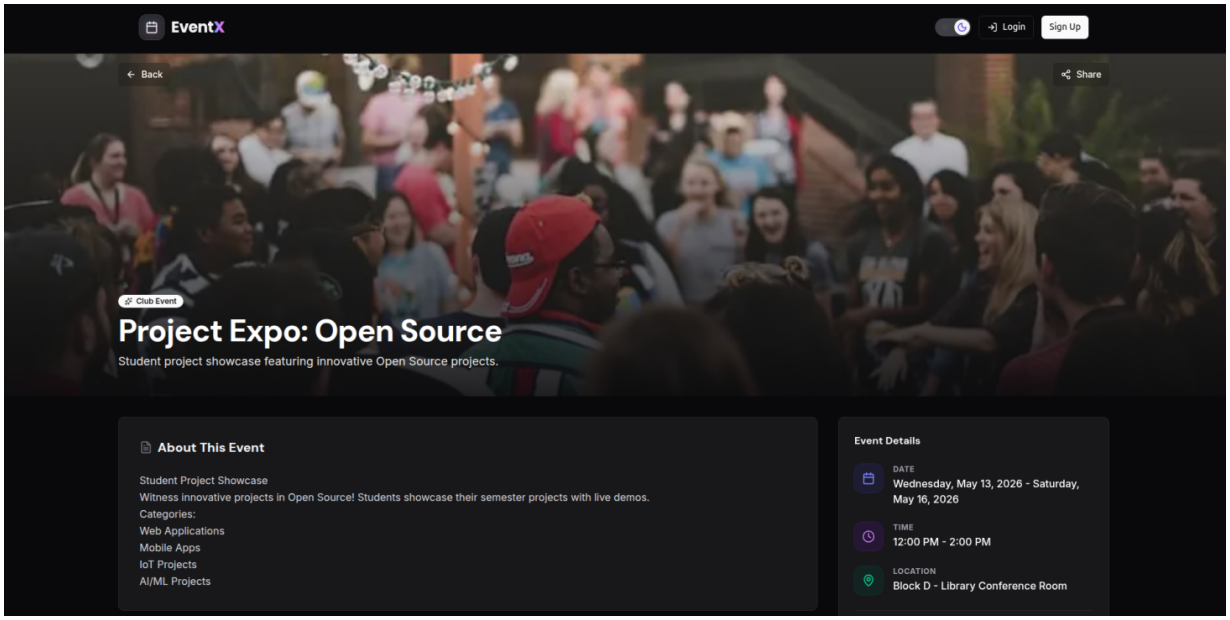


Figure 6.5: Event Details Page

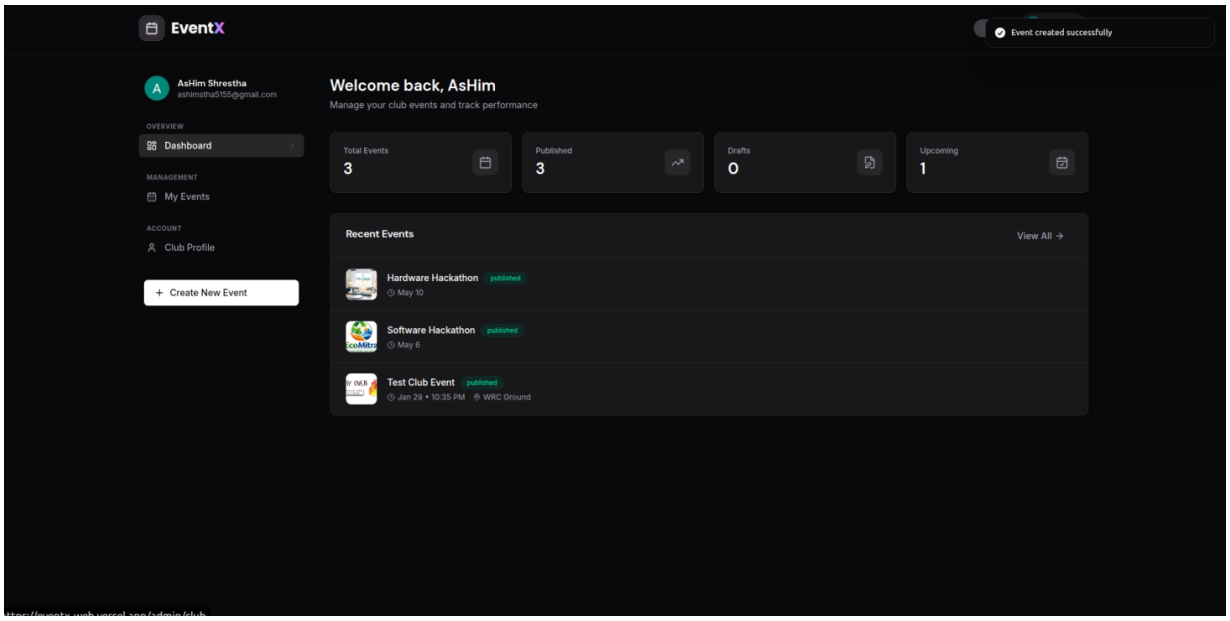


Figure 6.6: Club Admin Dashboard

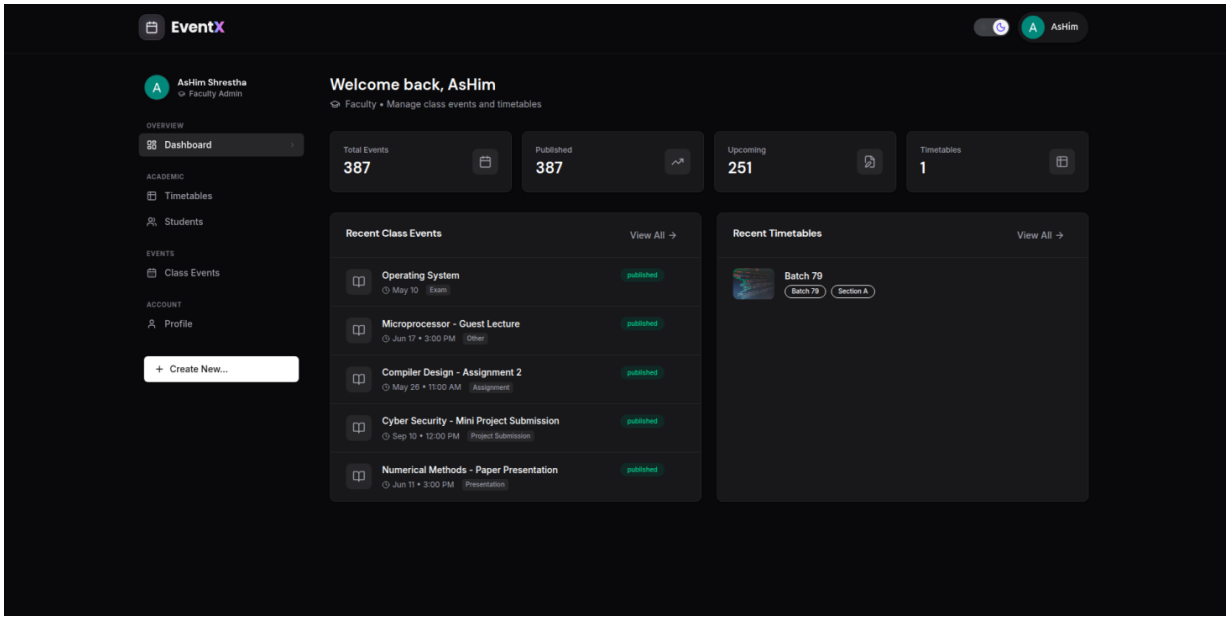


Figure 6.7: Faculty Admin Dashboard

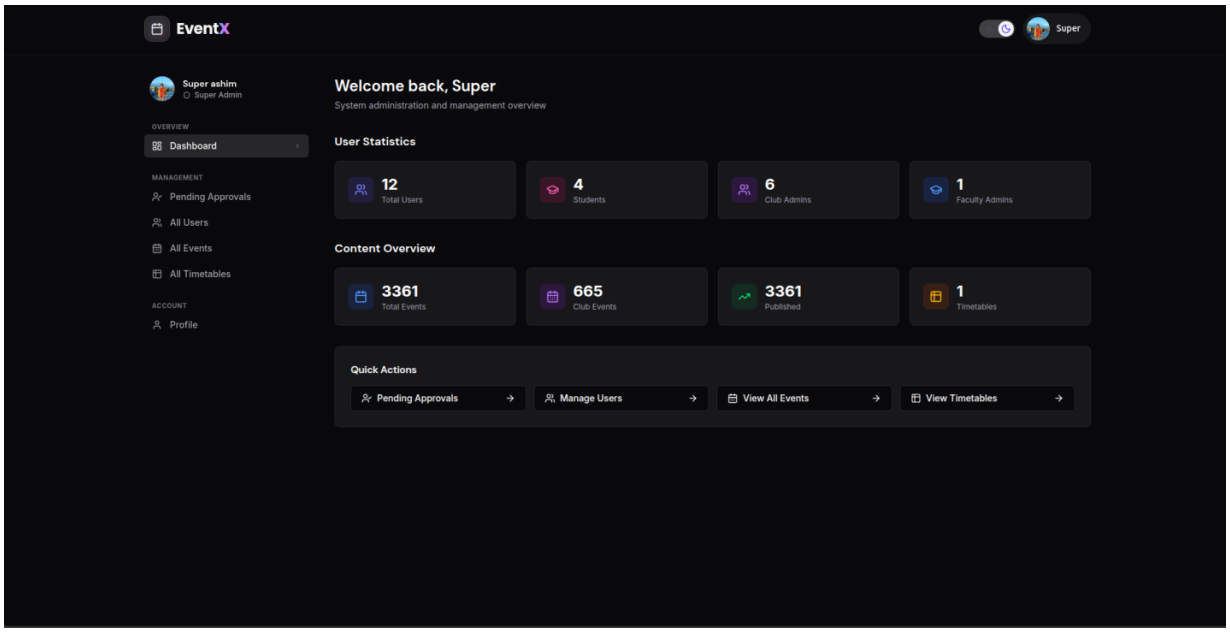


Figure 6.8: Super Admin Dashboard

7. RESULTS AND DISCUSSIONS

By including all the essential elements identified during the planning and analysis phase, the EventX project was able to achieve its main objectives. The system provides a centralized platform for managing academic, club, and institutional events in an organized and efficient manner. It helps students, faculty admins, club admins, and super admins interact with event-related information through a structured and user-friendly system.

The frontend of the application was designed to provide a simple, responsive, and easy-to-use interface where users can view event calendars, search for events, check event details, and receive important updates. The backend was implemented to handle user authentication, event creation, event approval, timetable management, notifications, and conflict checking in a reliable way. The database design supports the storage and management of users, roles, events, calendars, timetables, and notifications.

Comprehensive UML diagrams were created during the design phase according to Object-Oriented Analysis and Design (OOAD) principles. These diagrams included use case diagrams, class diagrams, sequence diagrams, activity diagrams, state diagrams, object diagrams, component diagrams, package diagrams, ER diagrams, and collaboration diagrams. These models helped define the structure, behavior, and interaction of the system clearly, making the application more scalable, maintainable, and understandable.

The key features of EventX were successfully analyzed and designed, including user login, role-based access control, event creation, event approval, event calendar viewing, event searching, timetable uploading, notification management, and event conflict checking. Faculty admins can manage class-related events, club admins can create club events, and super admins can approve or reject events before they are published. Students can view approved events, access timetables, and receive event notifications.

The project also focused on reducing manual effort and avoiding scheduling conflicts. By using a centralized event calendar and conflict-checking process, EventX helps prevent overlapping events and improves communication between students, faculty members, clubs, and administrators.

The project concluded with the successful delivery of EventX as a functional, reliable, and user-friendly event management system. It meets the objectives defined at the beginning of the project and provides an efficient solution for organizing college events. The system supports better planning, improved coordination, and easier access to event information, making it suitable for academic environments where multiple events need to be managed effectively.

8. FUTURE SCOPE

The EventX web application currently provides the essential features required for managing academic and extracurricular events within a college. However, the system can be further enhanced in several areas to improve usability and overall efficiency.

In the future, a dedicated mobile application for Android and iOS devices can be developed to provide easier and more convenient access to the platform. This would allow users to manage and view events more efficiently from their mobile devices.

Additional functionalities such as attendance tracking, QR-based event check-ins, participation analytics, and improved event approval workflows can further strengthen the system. Security can also be enhanced through advanced authentication methods and better permission management.

With continuous improvements and feature additions, EventX has the potential to become a more efficient and comprehensive platform for managing college events and improving coordination between students, faculty, and clubs.

9. CONCLUSION

The EventX web application provides a centralized and reliable platform to manage and view all college-related events in a single system. By presenting events in a calendar-based format, the application enables students to stay informed about upcoming academic, cultural, and extracurricular activities while ensuring that event information is easily accessible and well organized.

The system is designed to support multiple user roles, including students, faculty admins, club admins, and a super admin, allowing smooth coordination and effective management of events across the college. Features such as event filtering, notifications, reminders, and role-based administrative controls help improve communication, reduce scheduling conflicts, and increase student participation in campus activities.

With successful implementation, the EventX web application has created a more organized, transparent, and efficient event management environment. By streamlining event communication and improving accessibility, the system contributes to enhanced campus engagement and an improved overall college experience for students, faculty, and administrative staff.

10. EPILOGUE

The development of the **EventX** project has been a valuable and meaningful learning experience. From the initial planning phase to the final design and documentation, every step helped us understand how a real-world event management system can be analyzed, structured, and developed to solve practical problems in an academic environment.

Through this project, we explored important concepts related to system analysis, database design, user roles, event scheduling, notification management, and Object-Oriented Analysis and Design (OOAD). We learned how different users such as students, faculty admins, club admins, and super admins interact with the system, and how their requirements can be represented using UML diagrams and structured documentation.

One of the major learning outcomes of this project was understanding the importance of proper planning and system design before implementation. By preparing use case diagrams, class diagrams, sequence diagrams, activity diagrams, collaboration diagrams, ER diagrams, and other design models, we were able to clearly visualize the behavior, structure, and flow of the EventX system. This made the project more organized, scalable, and easier to understand.

The project also helped us realize the importance of communication and coordination in institutional event management. In many academic environments, events, timetables, and announcements are managed manually or through scattered platforms, which can lead to confusion and scheduling conflicts. EventX was designed to reduce these issues by providing a centralized platform where events can be created, approved, viewed, updated efficiently, while users can receive timely notifications.

Although the current version of EventX covers the main features such as event creation, approval, calendar viewing, timetable management, conflict checking, and notifications, there is still room for future improvement. Features such as real-time notifications, mobile application support, advanced analytics and integration with institutional systems could make the platform even more powerful and useful.

In the end, EventX was not just an academic project, but an attempt to design a practical solution for improving event coordination within an institution. This project encouraged us to think critically, apply software engineering concepts, and understand how technology can be used to simplify real-life administrative and academic activities. The experience gained from this project will be helpful for future development work and more advanced software engineering projects.

References

[1] C. Snehalatha and S. C. Nayagam, "Event management system," *International Journal of Engineering Research and Science & Technology*, 2025. [Online]. Available: https://ijerst.org

[2] "School event and announcement management system for student engagement," 2025, online resource. [Online]. Available: https://ithy.com

[3] R. Kumar *et al.*, "College club activity management system," arXiv, 2023. [Online]. Available: https://arxiv.org

[4] A. Chaturvedi, R. Sharma, and P. Verma, "A web-based event management system for academic institutions," *International Journal of Computer Applications*, vol. 185, no. 12, pp. 15–20, 2023. [Online]. Available: https://www.ijcaonline.org

[5] S. Patil, M. Desai, and A. Kulkarni, "Campus event management system using web technologies," *International Journal of Advanced Research in Computer Science*, vol. 13, no. 3, pp. 45–51, 2022. [Online]. Available: https://www.ijarcs.info

[6] D. Sharma and N. Gupta, "Real-time notification systems for student engagement in educational institutions," *International Journal of Information and Education Technology*, vol. 12, no. 5, pp. 389–396, 2022. [Online]. Available: https://www.ijiet.org

[7] Google, "Google calendar and calendar add-ons," 2024, accessed: 2024. [Online]. Available: https://developers.google.com/workspace/add-ons/calendar

[8] "College event management system," *International Journal of Scientific Research & Engineering Trends*, 2025. [Online]. Available: https://ijsret.com

[9] M. Raza, F. Ahmed, and J. Siddiqui, "Role-based access control in academic information systems: A systematic review," *Journal of Information Systems Engineering and Management*, vol. 6, no. 2, pp. 1–12, 2021. [Online]. Available: https://www.jisem-journal.com

[10] H. Ali, M. Rahman, and S. Hossain, "A unified digital platform for academic event scheduling and student participation," in *Proceedings of the International Conference on Computing and Communication Systems*, 2023, pp. 210–217. [Online]. Available: https://ieeexplore.ieee.org